TECHNICAL GUIDE

SAS to PySpark Migration Framework

Framework overview, supported conversion scope, design and architecture, algorithms, limitations, repository details, and validation examples.

Repository: chetanmbulla/Sas_to_pyspark_convertion

Documentation snapshot: 4 November 2026

Framework version: 0.1.0

Compatibility target: supported Base SAS subset to PySpark

1. Framework overview

`sas2pyspark` is a Python migration aid that translates a deliberately supported subset of SAS programs into reviewable PySpark source. It provides an installable Python package, command-line interface, Streamlit application, project inventory/conversion workflows, and optional Spark-based validation and reconciliation utilities.

The converter is conservative: unsupported or ambiguous syntax should fail with a `ConversionError` that identifies the source line and statement when available, rather than silently emitting a misleading translation. A successful conversion is not proof of semantic equivalence. Generated code must be reviewed and compared with representative SAS outputs before production use.

> **Primary objective:** reduce repetitive translation work while making assumptions, approximations, warnings, and unsupported constructs visible to migration teams.

Scope and intended use

- Translate supported Base SAS data-step transformations, selected procedures, expressions, macro constructs, date/format operations, and CSV input/output to PySpark.
- Support source-level conversion and a project workflow for discovering programs/data, recording conversions, reviewing outputs, and testing generated code.
- Help engineers build a migration baseline and identify unsupported syntax requiring manual conversion.
- Use Spark DataFrames and SQL temporary views as the generated runtime model.

Out of scope

- A complete SAS compiler, SAS runtime emulator, or automatic proof of business equivalence.
- Products and languages outside the documented subset, such as SAS/STAT, CAS, DS2, PROC IML, and arbitrary SAS/CONNECT behavior.
- Automatic access to SAS installations, licensed engines, SAS catalogs, or arbitrary `.sas7bdat` readers.

2. Functionality and capabilities

Area	Current capabilities
Entry points	Python API, `sas2pyspark` CLI, Streamlit conversion/testing UI, and project-oriented commands/workflows.
Parser and diagnostics	Source-aware lexical/statement parser with line locations; conservative conversion errors and warnings.
DATA steps	SET/WHERE, row filters, assignments, IF/ELSE, nested IF/DO and SELECT blocks, OUTPUT routing, MERGE BY and IN= flags, RETAIN variants, arrays, loops, inline DATALINES, _N_, LAG/DIF, and supported SET dataset options.
Procedures	PROC SQL, SORT, FREQ, MEANS/SUMMARY, UNIVARIATE approximation, TRANSPOSE, FORMAT, APPEND, DATASETS, IMPORT/EXPORT, PRINT, CONTENTS, COPY, and COMPARE within documented options.
Macros	%LET, macro variables, %MACRO calls and parameters, bounded nested/recursive expansion, compile-time %IF/%DO, text functions, %SYSFUNC subset, runtime variables, opt-in %SYSGET, and bounded dynamic %INCLUDE paths.
Dates and formats	SAS date-day and datetime-second counts, date/datetime literals, common date functions, supported informats/formats, VALUE/INVALUE/PICTURE definitions, and CNTLIN conversion for standard control columns.

Area	Current capabilities
Migration governance	Repository discovery, dataset/program dependency analysis, project conversion tracking, review/readiness reporting, local execution testing, and optional SAS/PySpark output reconciliation.

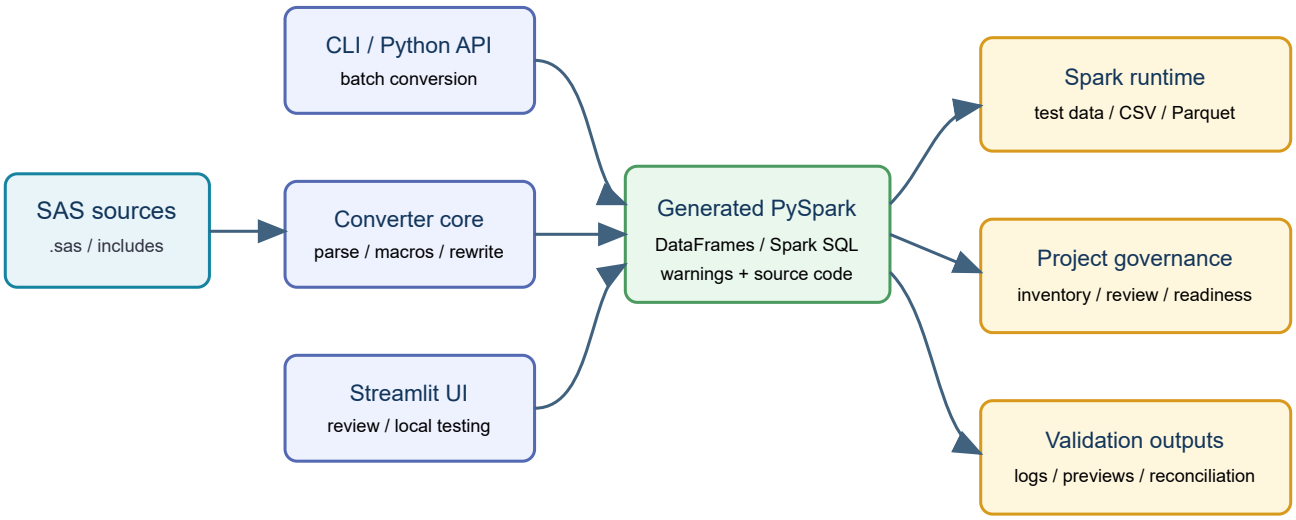
Recent committed work

Commit	Summary
`6f0e64d`	Runtime macro variables using PROC SQL SELECT INTO and CALL SYMPUT/SYMPUTX.
`5af224f`	Macro text functions, %SYSEVALF, %SYMEXIST, %RETURN, and forward %GOTO.
`c2f7221`	Broad converter extension covering blocks, dates, formats, includes, merge, outputs, RETAIN, procedures, macros, and SQL.
`6b72638`	Project restructuring and source-aware parser foundation.
`03b88cc`	Initial SAS-to-PySpark framework.

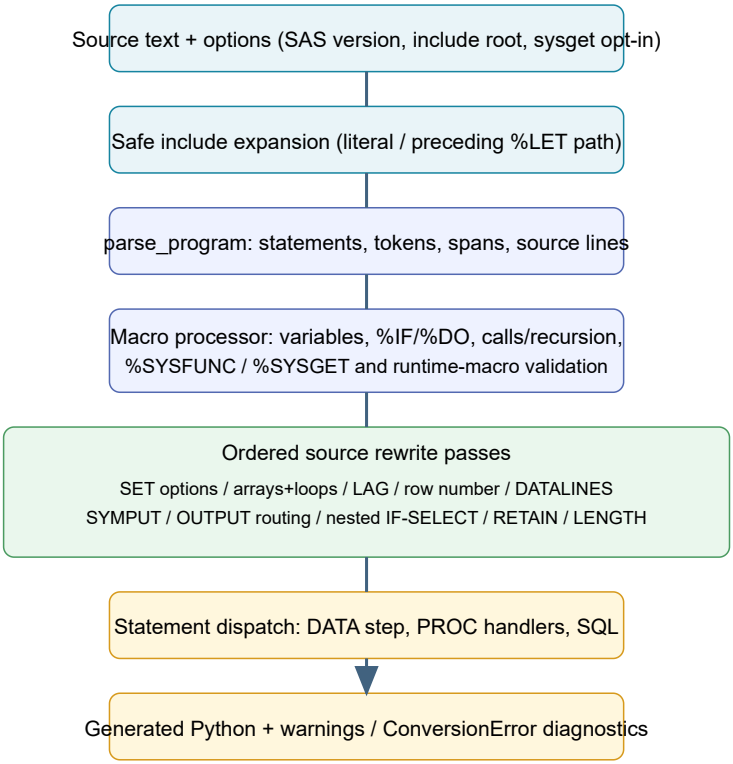
Worktree status at this documentation snapshot: implementation groups 1–6 include additional uncommitted changes for inline data/_N_, LAG/DIF and SET options, report procedures, nested data-step logic/arrays/RETAIN/LENGTH, SAS date/datetime semantics and format extensions, and macro/include enhancements. These changes are not represented by the commit list above. This guide and the README also include the group 7 documentation and local performance check; none of those uncommitted changes are represented by the commit list.

3. Design and architecture

High-level architecture



Low-level converter architecture



Process flow

1. Resolve source-version metadata and reset per-conversion format state.
2. Inline approved includes; enforce a root boundary, reject cycles, and limit nesting.
3. Parse SAS source into source-located statements and tokens.
4. Extract macro definitions and expand compile-time macro constructs under depth, loop, and output-size bounds.
5. Apply ordered prepasses for syntax requiring whole-step context and helper markers.
6. Dispatch statements to DATA-step and PROC converters; unsupported syntax raises an explicit error.
7. Return generated Python plus warnings, source-version selection, and diagnostics; user/runtime tools execute it separately.

Core algorithm

```
convert(source, options):
  reset per-conversion registries
  source = inline_safe_includes(source, include_root)
  statements = parse_program(source)
  statements = expand_macros(statements, allow_sysget)
  for rewrite in ordered_rewrite_passes:
      statements = rewrite(statements)
  output = generated_header
  while statements remain:
      if helper_marker: emit helper code
      elif DATA step: validate + translate data actions/windows
      elif supported PROC: validate + translate procedure
      else: raise source-located ConversionError
  return code, warnings, version profile
```

The ordered prepasses are intentionally explicit because transformations can depend on one another. The implementation uses named helpers in `src/sas2pyspark/`; see the low-level diagram for the current pipeline order.

4. Capabilities and semantic conventions

Data-step and procedure highlights

- Simple SET sources union by name; supported dataset options apply WHERE, FIRSTOBS/OBS, KEEP/DROP, then RENAME. MERGE BY approximates SAS match-merge behavior with windows and joins; per-input `ORDER_BY=(...)` options provide deterministic duplicate-key pairing.
- Nested block statements are converted to guarded assignments. IF/SELECT conditions that are unsafe to model are rejected.
- Output-producing indexed loops and statically bounded non-OUTPUT loops have different paths. The latter are unrolled under a statement cap.
- PROC PRINT maps to `DataFrame.show()`; CONTENTS displays a schema or column names and can emit Spark schema metadata with `OUT=`; COPY aliases views; COMPARE emits a limited summary.
- CSV IMPORT/EXPORT use Spark CSV readers/writers. PROC FORMAT supports documented static mappings and a bounded CNTLIN path.

`PROC CONTENTS OUT=` produces `NAME`, `SPARK_TYPE`, `NULLABLE`, and `POSITION` in Spark schema order. It is an explicit Spark-oriented metadata extension and does not reproduce the SAS output dataset schema.

Dates, missing values, ordering

- SAS date numerics are day counts from 1960-01-01. SAS datetime numerics are seconds from that date. These are not Spark DATE/TIMESTAMP columns.
- Character and numeric missing values map to Spark NULL with selected SAS-like string behavior. SAS special missing codes .A-.Z are not preserved as distinct states.
- Spark does not guarantee row order. By default monotonic IDs/windows are best-effort stand-ins for observation order in LAG/DIF, _N_, RETAIN, and BY flags. The `ORDER BY key1, key2 DESC;` DATA-step extension provides a checked unique key for deterministic SET-step windows; it does not sort output rows or affect SET FIRSTOBS/OBS. MERGE uses independent `ORDER_BY=(...)` keys on each input and validates uniqueness within BY groups.

5. Limitations and risk controls

Limitation	Impact / mitigation
Partial language and PROC coverage	Unsupported statements fail. Resolve manually or add a narrow converter feature with semantic tests.
No automatic SAS equivalence proof	Compare representative output rows, schemas, edge cases, and aggregates with real SAS runs.
Observation ordering	Without explicit ordering keys, Spark order is best effort. SET DATA steps require unique keys; MERGE requires a key on every input, unique within BY groups, and fails on missing or duplicate keys.
RETAIN and windows	Some RETAIN patterns require a single Spark partition or do not support BY/MERGE and self-reference. Avoid for large data without redesign.
Run-time macro variables	Generated Spark SQL session variables require Spark 4.0+ and are not usable in compile-time/structural contexts.
%SYSGET	Disabled by default. Opt-in reads host environment values during conversion; never expose secrets or untrusted values to conversion output.
Dynamic includes	Only paths resolvable from preceding %LET values are supported; macro-parameter includes are rejected. Path roots, cycles, and nesting are bounded.

Limitation	Impact / mitigation
CNTLIN formats	Rows are collected to the driver and should be small. Complex format catalogs/options are unsupported.
File formats and engines	CSV flows are supported; LIBNAME engines and general SAS datasets are not materialized. Supply data as Spark-readable files/DataFrames.
Execution security	Generated Python and the Streamlit test worker run with the user's account permissions; the worker is not a security sandbox. Run only trusted source/code.

6. Test scenarios and sample data

The automated suite covers source parsing, converter output, project workflows, governance/readiness, GUI testing, reconciliation, and Spark execution when the optional test dependencies and Java are available. At the documentation snapshot, the project test command is:

```
py -3.11 -m unittest discover -s tests
```

For direct local runs in this checkout, set `PYTHONPATH=src`. Spark integration tests use local Spark; PySpark 4 requires Java 17 or newer. The test extra is declared in `pyproject.toml`.

Representative scenarios

Scenario	Representative SAS	Input / expected behavior
Filter and assignment	`data out; set people; where age ge 18; adult=1; run;`	Rows below 18 are removed; retained rows receive `adult=1`.
Runtime macro variable	`proc sql noprint; select max(id) into :maxid from people; quit;`	Spark session variable is declared and available in later supported expressions (Spark 4+).
Nested macro / recursion	`%make(%identity(people));`, bounded countdown macro	Nested argument expansion works; recursion is bounded by the 32-level limit.
Dynamic include	`%let file=sub/part.sas; %include "&file.";`	Included source is resolved under the allowed root; traversal and cycles are rejected.
Date/datetime	`dt='02JAN1960:01:02:03'dt; d=datepart(dt);`	`dt=90123` seconds; `d=1` day; TIMEPART returns 3723 seconds.
INVALUE / PICTURE	`input(flag,yn.)`, `put(amount,dollars.)`	Exact mappings and supported digit mask produce mapped/rounded labels.
CNTLIN	`proc format cntlin=control; run;`	Small format control table maps score 1 to Low, 3 to Medium, 8 to High, and 12 to Other.
Nested blocks and arrays	`do i=1 to dim(q); q{i}=x*i; end;`	Static loops unroll; array element assignments map to element columns.

Sample input and program

```
people.csv
id,name,age
1,Ada,36
2,Bo,17
3,Cy,52
```

```
data work.adults;
  set work.people;
  where age ge 18;
  retain prior_name '';
  if age gt 40 then prior_name = name;
  band = 'adult';
  keep id name age prior_name band;
run;
```

Expected retained rows are Ada and Cy; Bo is filtered out. Because Spark row order is unspecified, production logic should use a stable explicit ordering key before relying on the retained value across multiple rows. Converter tests assert code shape and rejection behavior; Spark tests execute generated code with representative DataFrames and assert values, types, row counts, and helper-column cleanup.

Local performance check

A synthetic pipeline filtered `age >= 18`, derived `adult_flag` and `age_band`, then aggregated counts and flag sums. On Python 3.11.9, PySpark 4.3.0, Java 17.0.20.1, Spark `local[2]` with two shuffle partitions, warmed cached `Spark.range` inputs were measured three times:

Input rows	Output rows	Median elapsed
100,000	79,992	0.429 s
1,000,000	799,992	0.310 s

Conversion took about 7.84 ms and generated 384 characters. This is a local, synthetic, cached reproducibility snapshot, not a benchmark suite or a production throughput claim. The lower median for the larger input illustrates measurement noise from Spark warm-up, scheduling, and host load. Use representative uncached data and the target cluster for capacity or scaling decisions.

7. Repository details and operation

Repository	`https://github.com/chetanmbulla/Sas_to_pyspark_convertion`
Branch at snapshot	`main`
Package/version	`sas2pyspark` / `0.1.0`
Python support	Python 3.10+ per `pyproject.toml`; validation environment Python 3.11.
Dependencies	Core converter has no third-party runtime dependencies. Optional extras provide PySpark, Streamlit, and reconciliation packages.
Key locations	`src/sas2pyspark/` package; `app/` UI; `tests/`; `examples/`; `docs/`.

Quick commands (PowerShell)

```
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e .
.\.venv\Scripts\sas2pyspark.exe input.sas --output output.py
py -3.11 -m unittest discover -s tests
```

Python API: `from sas2pyspark import SasToPySparkConverter`, then `SasToPySparkConverter().convert(source)`. Enable environment access explicitly with `SasToPySparkConverter(allow_sysget=True)`. The CLI and GUI do not enable

%SYSGET by default.

8. Release and validation guidance

1. Inventory the SAS programs, input/output datasets, libraries, dependencies, and available SAS reference outputs.
2. Convert a small representative program; inspect warnings and errors instead of treating code generation as a pass/fail equivalence check.
3. Review generated Python and confirm the Spark schema, null behavior, ordering, date types/units, and side effects.
4. Run deterministic test cases and compare outputs with SAS using the repository's reconciliation workflow where feasible.
5. Record unresolved syntax and semantic gaps; keep unsupported logic blocked from production until manually translated and validated.

This guide documents repository behavior and engineering conventions as of 4 November 2026. Feature support can change; consult the current README, tests, and source before relying on a particular construct.